Unit-1 Introduction to Python

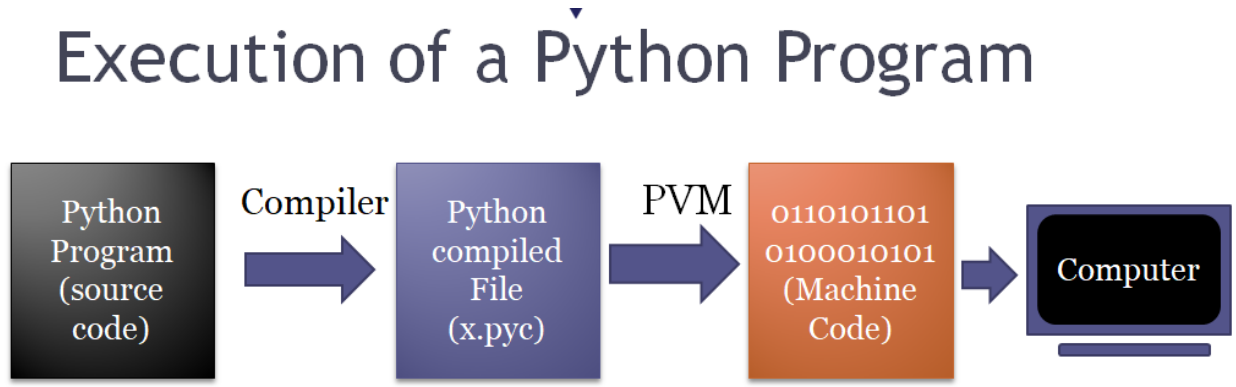
Features of Python

1). Simple 2). Easy to learn 3). Open source 4).High level language
5). Dynamically typed 6). Platform independent 7). Portable

(1).Simple:
Python is a simple programming language. When we read a python program, we fill like reading English sentences. Hence, understanding and developing programs becomes easy.
(2). Easy to learn:
The structure of python program is very simple. Python uses very few keywords. Hence learning and developing programs in python become easy.
(3). Open Source:
Python is freely available from www.python.org, it can be downloaded and use. There is no need to pay for python software.
(4). High level language:
Programming languages are of two types: low level and high level. A low level language uses machine code instructions to develop program. Python is a high level programming language as it uses English words to develop programs.
(5). Dynamically typed:
In python, we need not declare anything. An assignment statement binds a name to an object, and the object can be of any type.
(6). Platform independent:
After the compilation of the python program, it generates byte code. The byte code is fixed set of instructions that run on all operating systems.
(7). Portable:
As we know that python code are platform independent, its code will give same result in any computer in the world.

Execution of a Python Program

Suppose that we have a python program named first.py. Here, 'first' is the program name and the .py is the extension of python code file.
After writing the code, the next step is to compile the code. The compiler converts the python program into the byte code.
Byte code represents a fixed set of instructions that represents all operations, which run on any operating system and hardware. It means that byte instructions are system independent and platform independent.
The computer cannot execute the byte code. So, it is necessary to convert byte code to machine code.
PVM(Python Virtual Machine) interprets the byte code and convert it to the machine code.



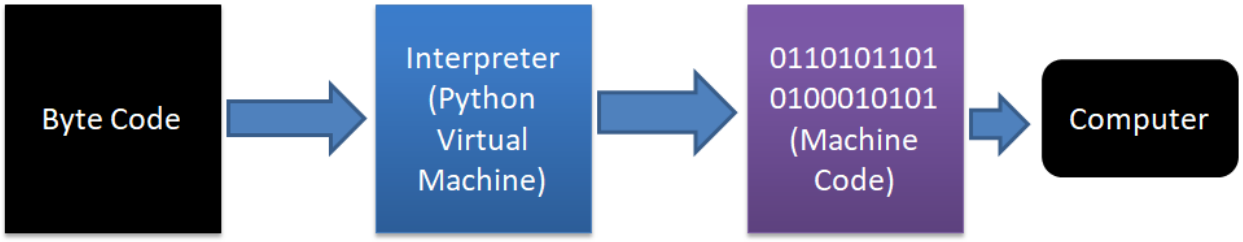
Normally, after compiling the code the .pyc file is generated. That is converted to machine code using PVM. But we cannot see .pyc file or machine code generated by PVM. It is done internally.
If we want to create a .pyc file and see it, we must use following command:
>>> python –m py_compile first.py
-m option represents the module and module name is py_compile.
This module generate .pyc file of the first.py
The .pyc file name would consist something like cpython, which indicates that we are using the python compiler that was created using C language.
The compiler created by C language is considered as a standard compiler.
The file name would be something like: helloworld.cpython-311, run it on python terminal like below:
helloworld.cpython-311.pyc
You will get the output immediately.

Viewing the Byte code

We want to see the byte code instructions that were created internally by the python compiler before they are executed by the PVM. To view the byte code of the python program>>>python –m dis first.py

Python Virtual Machine (PVM)

Since computer understands only machine code, it is obvious that we should convert any program into machine code before it is submitted to the computer for execution. A compiler normally converts the program source code into machine code.
A Python compiler does the same task but in a slightly different manner. It converts the program source code into byte code (in the .pyc format).



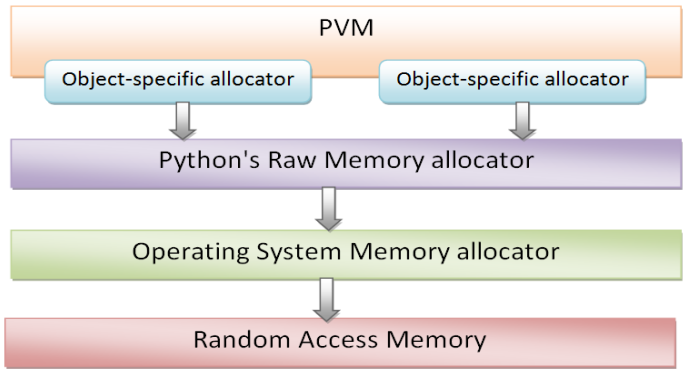
The role of PVM is to convert the byte code instructions into machine code so that the computer can execute those machine code instructions and display the final output. A PVM is equipped with an interpreter which converts the byte code into machine code and sends it to the computer processor for execution. Since an interpreter plays an important role, often PVM is also known as interpreter.

Memory management

In the programming language like C and C++, the programmer needs to allocate and de-allocate memory dynamically during runtime.
In Python, memory allocation and de-allocation are done during runtime automatically. Python's PVM will take care of such issues.

A python considers string, lists, functions and even modules as an object. The memory manager inside the PVM allocates memory required for objects created in a python program. All this objects are stored in the separate memory called heap. Heap is the memory which is allocated during runtime. The size of the heap memory depends on the RAM (Random Access Memory), which can increase or decrease its size depending on the requirement of the program.

The operating system will help to allocate the memory in the RAM.
A raw memory allocator oversees whether memory is available to it for storing objects.
On the top of the raw memory allocator, there are several object-specific allocators which operate on the same heap. These object-specific allocators do the task of memory management depending upon the type of the objects.



Comparisons between Java and Python

Java	Python
Java is object oriented programming language. Functional programming features are introduced into Java 8.0	Python blends the functional programming with object oriented programming features.
Java code requires more number of lines.	Python programs are concise and compact.
It is compulsory to declare the data types of variables, arrays etc. In Java.	Type declaration is not required in Python.
Java has do... while, while, for and for each loops.	Python has while and for loops.
Java has switch statement.	Python does not have switch statement.
Memory allocation is done automatically by JVM (Java virtual Machine).	Memory allocation is done automatically by PVM (Python virtual Machine).
Java supports single and multi-dimensional arrays.	Python supports only single dimensional arrays. To work with multi-dimensional arrays, we should use third party applications like numpy.
The array index should be a positive integer.	Array index can be positive or negative integer number. Negative index represents locations from the end of the array.
A semicolon is used to terminate the statements and comma is used to separate expressions.	New line indicates end of the statements and semicolon is used as an expression separator.

Comments in Python

Comments are of two types: 1. Single line comment and 2. Multi line comment

<u>Single line comment:</u>
The single line comment uses hash symbol (#) at the starting of a line. The entire line after the hash symbol will be considered as a comment.

```
#The below line will print Atmiya Uniiversity
Print('Atmiya University')
```

Comments are non-executalbe statements. It means neither the python compiler nor the PVM will execute them.

<u>Multi line comment:</u>

When it is required to mark more than one line as a comment, writing # symbol in the beginning of every line is tedious.
Instead of writing # at starting of every line, we can use (“”“) or (‘’’) before and after the lines to be commented.

```
"""
Take name from the user
Take contact details of the user
"""
```

[or]

```
'''
Take name from the user
Take contact details of the user
'''
```

Garbage Collection in Python

Garbage collection is a module in Python that is useful to delete objects from memory which are not used in the program. The module of Garbage collection in Python is known as 'gc'. Garbage collector maintains the count of each object regarding number of times it is referred (or used). When an object is used 3 times than its reference count will be 3. Count greater than 0 means an object is being used or referred to. When an object is found with count 0, garbage collector will free the memory allocated to that object.

Garbage collector runs automatically. But in the case, when more number of objects is created and if the system runs out of memory, then automatic garbage collector will not run. In this case, manual garbage collection is to be used. Manual garbage collection is of two types: time based manual garbage collection and event based manual garbage collection.

Time based manual garbage collection: If the garbage collector is called in certain intervals of time, it is called time based manual garbage collection.

Event based manual garbage collection: If the garbage collector is called on the basis of event, for example, when the user disconnects from an application, it is called event based garbage collection.

Garbage collector if called for running frequently will slow down the program execution.

How Python sees Variables

In Programming languages like C, Java, etc. the concept of variable is connected to memory location. This kind of programming languages variable is used as a storage box.

Suppose that we write a =7, then 'a' is allocated with some memory and 7 is stored into it.

Now suppose we change the value of 'a', a=34 then it may be visualised as below:

When we assign one variable to another variable then it may be visualised as below:

But, in Python variable is seen as a tag (or name) that is tied to some value.

For example a=7 in python would be like:

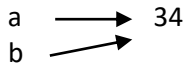
a ⟶ 7

Now suppose, we change the value of 'a', a=34, then the tag is simply changed to the new value (or object).

a ⟶ 34 7

Assigning one variable to another variable makes a new tag bound to the same value.

For example a=b



Python has 'tags' (or names) to represent the values. Python is using memory efficiently.

Datatypes in Python

A datatype represents the type of data stored into a variable or memory.

Built-in datatypes:
1). None type
2). Numeric types
3). Sequences
4). Sets
5). Mappings

(1) <u>None type:</u>
In Python, the 'None' datatype represents an object that does not contain any value. In Python, it is called 'None' object.
Python allows only one 'None' object in a program. 'None' can be used inside a function as a default value of the arguments. While calling a function, if no value is passed, then the default value will be taken as 'None'.

(2) <u>Numeric type: (int, float, complex):</u>
<u>Int datatype:</u>
An int datatype represents an integer number. The number must be without decimal or fractional values. Values can be either positive or negative; it can store very large integer value easily. Example: a=-34, b=1246753876
<u>float datatype:</u>
A float datatype represents a floating point number. The number must be with decimal or fractional values. It also accepts number in scientific notation where we use 'e' or 'E' to represent the power of 10. For example $3.4*10^3$ is written as $3.4e^3$. Suppose we write like a=3.4e3, then $3.4*10^3$ is stored in 'a'.
<u>Complex datatype:</u>
A complex number is a number that is written in the form of a+bj. Here 'a' represents the real part of the number and 'b' may represent the imaginary part of the number. The part 'a' and 'b' may contain integer or float value. For example: 3+5j, 0.2+10.5j.

```
a=2.5+2.5j
b=3.0-0.5j
c=a+b
print("The sum is", c)
```

<u>Representing Binary, Octal and Hexadecimal Numbers</u>
A binary numbers are written by prefixing 0b (zero and b or B) before the value.
Example: 0b10111010
Octal numbers are written by prefixing 0o (zero and o or O) before the value.
Example: 0o214
Hexadecimal numbers are written by prefixing 0x (zero and x or X) before the value.
Example: 0xD214

A Python program to convert number to decimal number

```
no1=0b10111010
no2=0o214
no3=0xD214
i=int(no1)
print('The decimal of 0b10111010 is:', i)
i=int(no2)
print('The decimal of 0o214 is:', i)
i=int(no3)
print('The decimal of 0xD214 is:', i)
```

There is another way too:

```
no1=10111010
no2=214
no3=D214
i=int(no1,2)
print('The decimal of 10111010 is:', i)
i=int(no2,8)
print('The decimal of 214 is:', i)
i=int(no3,16)
print('The decimal of D214 is:', i)
```

We can also convert a number to binary using bin(), to octal using oct() and to hexadecimal using hex().

```
no=7
binary = bin(no)
print (binary)
octal = oct(no)
print (octal)
hexadecimal = hex(no)
print (hexadecimal)
```

Converting the datatypes explicitly

Python assumes the datatype for the variable depending upon the type of data. If somebody wants to convert the type by his own, it is called type conversion.

```
#Convert the number into integer
a=34.7
b=int(a)
print(b)
#Convert the number into float
a=34
b=float(a)
print(b)
```

Boolean datatype

Boolean datatype represents Boolean value True and False

```
no=7<34
print(no)
```

```
no=7>34
print(no)

a=True+True
print(a)
a=True+False
print(a)
```

(3) Sequences

Sequence represents group of elements or items. A group of integer numbers will form a sequence.

Str Datatype

A string is represented by a group of characters which are enclosed in single quotes or double quotes. Example: “Atmiya” or ‘Atmiya’. String can also written inside “”” (triple double quotes) or ‘’’(triple single quotes) to span group of lines including spaces.

Example:

```
a=”””Python is one of the very interesting
     programming language, which requires
     very less code.”””
                    [OR]
b=’’’ Python is one of the very interesting
     programming language, which requires
     very less code. ’’’
```

To retrieve piece of string from the whole string square[] brackets are used. The count of string begins with ‘0’.

Example:

```
a="Welcome to Atmiya University"
print(a[0])
print(a[11:21])
print(a[11:])
print(a[-1:])
print(a[-1:-11])
```

We can also use repetition operator ‘*’. Print(a*2)

Bytes datatype

The bytes datatype represents a group of byte numbers just like an array does. A byte number can be any positive number between 0 to 255. We cannot modify any element from bytes.

Example:

```
a=[2,7,11,34,16,25] # This is a list of a byte numbers
b=bytes(a) # Convert the list into bytes array
print(b[0]) # Displays 0th element
b[1]=8 # It will give an error
```

bytearray datatype

Bytearray datatype is similar to bytes datatype, with one difference. The difference is that, we can modify the elements of bytearray.

Example:

```
a=[2,7,11,34,16,25] # This is a list of a byte numbers
b=bytearray(a) # Convert the list into bytes array type array
print(b)
```

```
print(b[0]) # Displays 0th element
print(b[1]) # Displays 1st element
b[1]=43 # It will replace 7 with 43
print(b[1])
```

list datatype

List in Python is similar to arrays in C or Java. A list represents a group of elements. The difference between list and array is that a list can store different types of elements but an array can store only one type of elements. The size of array is fixed but list can grow dynamically in memory. Elements of lists are written between [ ]. Elements of list can be modified.

Example:

```
a=['Atmiya', "University", 'Rajkot', 7, -45, 23.2]
print(a)
print(a[0])
print(a[0:3])
print(a[-1])
print(a*2)
a[2]="Gujarat"
print(a)
```

tuple datatype

A tuple contains a group of elements which can be of different types. The list and tuple are same with two differences. First difference is that, elements of tuple are written between parenthesis ( ). And the second difference is, elements of tuple cannot be modified. Elements of tuple cannot be modified.

Example:

```
a=('Atmiya', "University", 'Rajkot', 7, -45, 23.2)
print(a)
print(a[0])
print(a[0:3])
print(a[-1])
print(a*2)
a[2]="Gujarat" # we cannot modify elements of tuple
print(a)
```

range datatype

The rang datatype represents a sequence of numbers. The numbers in the range are not modifiable. Generally, range is used for repeating a 'for loop' for a specific number of times.

Example:

```
a=range(8) # I will print range of numbers from 0 to 7, 8 not included
for i in a:print(i)
a=range(11,44,2) # It will print range starting form 11 with 1 jump every time till 43
for i in a:print(i)
a=list(range(8))
print(a)
print(a[0])
```

(4) Sets

A set is an unordered collection of elements in which order of elements is not maintained. It means the order in which output is generated may not be same as the input. A set does not

accept the duplicate elements. We can add or remove elements to/from the set. We cannot retrieve any element from the set as it does not generate set in the same order.

Set datatype

Elements in the set must be entered comma separated between { } (curly brackets).

Example: Creating the set

```
a={7,16,52,34,43,43}
print(a) # Compare the output and input
```

While we observe the output generated, we can see that the sequence in which elements were entered in the set are changed when we print the output. The other significant thing to observe is that, we have entered 43 two times but when we printed the set 43 appeared only for one time. It means set has stored 43 only one time.

We can use set function like set(), to create as set as:

Example: Creating set of string

```
a=set("Atmiya")
print(a)
a=set("AAtmiyaa")
print(a)
```

We can convert list to set as below:

Example: Creating the set using list

```
lst=[7,16,52,34,43,43]
a=set(lst)
print(a)
```

We can add elements to the existing set using update().

Example: Adding elements to the existing set

```
a.update([20,11])
print(a)
```

We can remove element from the set using remove(). Remove() can remove only one element at a time.

Example: Removing element from the list

```
a.remove(11)
print(a)
```

frozenset datatype

The frozenset datatype is same as the set datatype with one difference that data cannot be modified. We can create frozenset by passing a set to the frozenset().

Example: Creating frozenset

```
a={7,16,52,34,43,43}
print(a) # Set created
# Converting the set to frozenset
fs=frozenset(a)
print(fs)
```

We can create a frozenset by passing a string to the frozenset() function.

Example: Creating frozenset using string

```
a=frozenset("Atmiya")
print(a)
```

(5) Mapping Types (dictionary)

A map represents group of elements in the form of key and value pairs. While retrieving the value, when the key is given value associated to it is retrieved. The key and its associated value should be separated by a colon and every pair must be separated by a comma. All the elements must be enclosed inside curly brackets.

Example: Creating the dictionary

```
a={1:"ABC", 2:"CDE", 3:"EFG", 4:"GHI", 5:"IJK"}
print(a)
```

We can create an empty dictionary without any element. Data can be added into the created Later on.

Example:

```
b={} # Empty dictionary created
#Storing key and values into b
b[1]="ABC"
b[2]="CDE"
b[3]="EFG"
print(b)
```

We can retrieve value, using key b[key] as below:

```
print(b[3])
```

Retrieving only keys can be done using keys():

```
print(b.keys())
```

Retrieving only values can be done using values():

```
print(b.values())
```

We can update the value of any particular key.

```
b[2]="XYZ"
print(b)
```

We can delete and element.

```
del b[2]
print(b)
```

Literals in Python

A constant value stored in the variable in a program is known as literal.

Example: a=25

Here 'a' is the variable in which constant value 25 is stored, 25 is called literal. As 25 is an integer value, it is called 'integer literal'.

Types of literals found in Python are as follow:

-Numeric literals
-Boolean literals
-String literals

<u>Numeric literals</u>

Available numeric literals in Python are as below:

Example	**Literal name**
340, -34	Integer literal
3.22, -3.22, 1.24E7	Float literal
0x25A	Hexadecimal literal
0241	Octal literal
0B110110	Binary literal

16+7J	Complex literal

Boolean literals

Boolean literals are in the form of 'True' or 'False' values stored into a Boolean type variable.

String literals

A group of characters is called a string literal. These string literals are enclosed in single quotes or double quotes or triple quotes. In Python there is no difference between single quoted strings or double quoted strings.

```
a='Python is easy'
b="Python is easy"
print(a)
print(b)
```

When a string literal goes beyond the single line, triple quotes must be used.

```
a="""Python is very helpful programming language
It is very trending and highly used language."""
print(a)
```

We can use escape characters as required inside a string literal.

Escape character	Meaning
\	New line continuation
\n	New line
\\	Display single \
\'	Display single quote
\"	Display double quote
\b	Backspace
\r	Enter
\t	Horizontal tab space
\v	Vertical tab space

Using backslash(\ ) will join the content of next line to the line.

```
a="""Python is very helpful programming language \
it is very trending and highly used language."""
print(a)
```

Using (\n) will take the text to the new line.

```
a="""Python is very helpful \n programming language \
it is very trending and highly used language."""
print(a)
```

Determining the datatype of a variable

We can use the type() to know the datatype of a variable or object.

```
a=16
print(type(a))
```

Since we are stroing 16 into 'a', the type of 'a' is assumed by the Python interpreter as 'int'. It means that variable 'a' is an object of the class 'int'. In fact, every datatype, function, method, class, module, lists, sets are all objects in Python.

```
a=12.4
print(type(a))
a="Atmiya"
```

```
print(type(a))
a=[1,2,3,1]
print(type(a))
a=(1,2,3,1)
print(type(a))
a={1,2,3,1}
print(type(a))
```

Characters

Python does not contain char datatype to represent individual characters It has string (str) dataype which represents strings. As a result, when we think of a character, we should think of it as an element in a string.

```
a='A'
print(type(a))
```

We can access the individual characters of a string using index or position number.

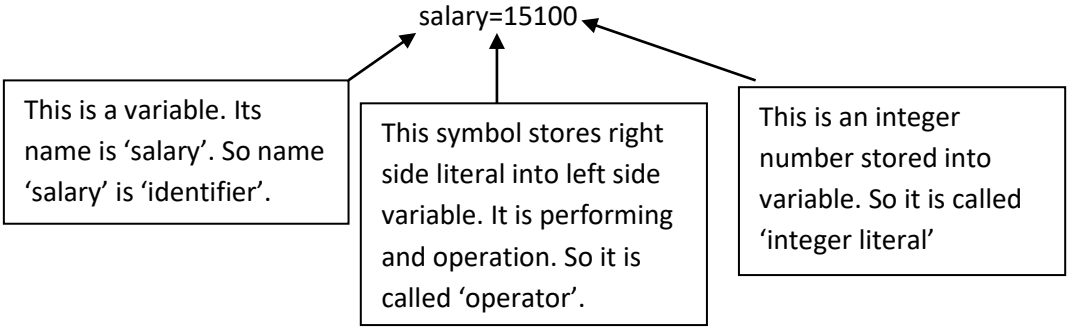
```
a="Atmiya"
print(a[0])
```

Constants in Python

A constant is similar to a variable but its value cannot be modified during program execution. Value of normal variable can be changed when required, which is not possible for a constant. Once defined, a constant allow changing its value. For example, value of 'pi' in mathematics is 22/7 which never changes. Python does not allow creating a constant value. A programmer can indicate a variable as constant by writing its name in all capital letters. For example, MAX_VALUE is a constant. But its value can be changed.

Identifiers and Reserved words

An identifier is a name that is given to a variable, function, or a class etc. Identifiers can include letters, numbers, and the underscore (_). Identifiers should not start with numeric values. Use of other special characters is not allowed. As Python is a case sensitive programming language, it means capital letters and small letters are identified as separate by Python. Example: a=16 and A=16 are two different identifiers.



Reserved words are the words that are already reserved for some particular purpose in Python language. These reserved words should not be used as identifiers.

and	del	from	nonlocal	try
as	elif	global	not	while
assert	else	if	or	with
break	except	import	pass	yield
class	exec	in	print	False
continue	finally	is	raise	True
def	for	lambda	return	

Naming conventions in Python

The rules related to writing names of variables, packages, modules, classes, etc. are called naming conventions. Following naming conventions should be followed:

Packages	Names must be written in all lower case letters. When multiple words are used for an name, we should separate them using an underscore (_).
Modules	
Global variables or Module level variables	
Instance variables	
Functions	
Methods	
Constants	Names must be written in all upper case letters. When multiple words are used for an name, we should separate them using an underscore (_).
Classes	Each word of a class name should start with a capital letter. Whereas, Pythons built-in class names use all lower case words. When a class represents exception, then its name end with a word 'Error'.
Non public instance variable	Name should begin with an underscore.
Method arguments	In case of instance methods. Their first argument name should be 'self'. In case of class methods, their first argument should be separated by an underscore (_).
Non accessible entities	Some variables, functions and methods are not accessible outside and they should be used as they are in the program. Such entities names are written with two underscore before and two underscore after. For example, __init__(self) is a function used in a class to initialize variables.

Operator

An operator is a symbol that performs an operation. An operator acts on some variables which are known as operands. We can classify the operators depending upon their nature, as below:

1). Arithmetic operators
2). Assignment operators
3). Unary minus operators

4). Relational operators
5). Logical operators
6). Boolean operators
7). Bitwise operators
8). Membership operators
9). Identity operators

(1). Arithmetic operators

This type of operators is used to perform basic arithmetic operations.

Operator	Meaning
+	Addition operator. Adds two values.
-	Subtraction operator. Subtracts one value from another.
*	Multiplication operator. Multiplies two values.
/	Division operator. Divides left operand by the right operand.
%	Modulus operator. Gives remainder to division.
**	Exponent operator. Calculates exponential power value.
//	Integer division. Performs division and gives only integer quotient.

When an expression contains several arithmetic operators, then the evaluation is done in following order:

Parenthesis are evaluated
Exponent is done
Multiplication, division, modulus and floor divisions are done at equal priority
Addition and subtraction
Finally assignment operator is performed.

(2). Assignment operators

Generally, these types of operators are used to store the right side value to the left side variable. We can also use these operators to perform some simple arithmetic operations and store it to the variable.

It is possible to assign the same value to two variables:

```
a=b=7
print(a,b)
```

It is possible to assign different values to two variables:

```
a,b=7,6
print(a,b)
```

Operator	Example	Meaning	Result
=	a=b+c	Assignment operator. Stores right side value into left side variable. It means, the answer of addition of b and c is stored into a.	18
+=	a+=b	Addition assignment operator. Adds right operand to the left operand and stores the result into left operand. Like a=a+b	18
-=	a-=b	Subtraction assignment operator. Subtracts right operand to the left operand and stores the result into left operand. Like a=a-b	14
*=	a*=b	Multiplication assignment operator. Multiplies right operand to the left operand and stores the result into left operand. Like a=a*b	32
/=	a/=b	Division assignment operator. Divides left operand with right operand and stores the result into left operand. Like a=a/b	8.0
%=	a%=b	Modulus assignment operator. Divides left operand with right operand and stores the result into left operand. Like a=a%b	0
**=	a**=b	Exponentiation assignment operator. Performs power value and then stores the result into left operand. Like a=a**b	256
//=	a//=b	Floor division assignment operator. Performs floor division and then stores the result into left operand. Like a=a**b	8

Here the assumed values are a=16 and b=2.

(3). Unary Minus Operator

The unary minus operator is denoted by minus (-), when the operator is used before a variable. By doing so the value of variable is negated. It means, if the value of a variable is positive than it will converted to negative and if its value is negative than it will converted to positive.

Example:

```
a=7
print(-a)
```

(4). Relational Operator

Relational operators are used to compare two values or variables, whether values in both variables are same or which one is bigger or which one is lesser. These operators give result into True or False.

Operator	Example	Meaning	Result
>	a>b	Greater than operator. If the value of left operand is greater than the value of right operand, it gives True else False.	True
>=	a>=b	Greater than or equal operator. If the value of left operand is greater than or equal to the value of right operand, it gives True else False.	True
<	a<b	Less than operator. If the value of left operand is lesser than the value of right operand, it gives True else False.	False
<=	A<=b	Less than or equal operator. If the value of left operand is lesser than or equal to the value of right operand, it gives True else False.	False
==	a==b	Equals operator. If the value of left operand is equal to the value of right operand, it gives True else False.	False
!=	a!=b	Not equals operator. If the value of left operand is not equal to the value of right operand, it gives True else False.	True

Here the assumed values are a=16 and b=2.

(5). Logical Operator

Logical operators are useful to construct compound conditions. A compound condition is a combination of more than one simple condition. Each of the simple condition is evaluated to True or False and then the decision is taken to know whether the total condition is True or False. In case of logical operators, False indicates 0 and True indicates any other number.

Operator	Example	Meaning	Result
and	a and b	If a is False, it returns a, otherwise it returns b.	200
or	a or b	If a is False, it returns b, otherwise it returns a.	100
not	not b	If a is False, it returns True, otherwise False.	False

```
#If a is False, it returns a, otherwise it returns b.
        a=100
        b=200
        print(a and b)
#If a is False, it returns a, otherwise it returns b.
        a=0
        b=200
        print(a and b)
#If a is False, it returns b, otherwise it returns a.
        a=100
        b=200
        print(a or b)
#If a is False, it returns b, otherwise it returns a.
        a=0
        b=200
        print(a or b)
If a is False, it returns True, otherwise False.
        a=100
        b=200
        print(not a)
If a is False, it returns True, otherwise False.
        a=0
        b=200
        print(not a)
a=100
b=200
c=300
#Using 'and' with if condition
if(a<b and b<c):print('Yes')
else: print('No')
#Using 'or' with if condition
if(a>b or b<c):print('Yes')
else: print('No')
```

(6). Boolean Operators:

The result provided by Boolean operators will either True or False.

Operator	Example	Meaning
and	a and b	Boolean and operator. If a and b are True, then it returns True, otherwise False.
or	a or b	Boolean or operator. If either a or b is True, then it returns True, otherwise False.
not	a not b	Boolean not operator. If a is True, it returns False, else True.

```
a=True
b=False

a and a
a and b
a or a
b or b
not a
not b
```

(7). Membership operators

The membership operators are useful to test for membership in a sequence such as strings, lists, tuples or dictionaries. For example, if an element is found in the sequence or not can be asserted using these operators. There are two membership operators:

-in

-not in

The in Operator

This operator returns True if an element is found in the specified sequence. If the element is not found in the sequence, then it returns False.

```
"A" in "Atmiya" # in the python prompt
flowers=["Mogra","Rose","Sunflower","Marigold"]
print("Mogra" in flowers)

flowers=["Mogra","Rose","Sunflower","Marigold"]
print("mogra" in flowers)

flowers=["Mogra","Rose","Sunflower","Marigold"]
print("mogra" not in flowers)

flowers=["Mogra","Rose","Sunflower","Marigold"]
print("Mogra" not in flowers)

city=('Rajkot','Ahmedabad','Surat','Vadodara')
```

```
print('Rajkot' in city)
```

<u>The not in Operator</u>

This operator returns True if an element is not found in the sequence. If the element is found in the sequence, then it returns False.

<u>(8). Identity operators</u>

These operators compare the memory locations of two objects. The memory location of an object can be seen using the id() function. This function returns an integer number, called the identity number that internally represents the memory location of the object.

Example: in the python prompt

```
a=16
b=16
id(a)
id(b)
```

There are two identity operators:

-is

-is not

The 'is' and 'is not' operators do not compare the values of the objects. They compare the identity numbers or memory locations of the objects.

<u>The 'is' Operator</u>

The 'is' operator is useful to compare whether two objects are same or not. It will internally compare the identity number of the objects. If the identity numbers of the objects are same, it will return True; otherwise, it returns False.

Example:

```
a=16
b=16
if(a is b):print('a and b have same identity')
else:print('a and b do not have same identity')
```

Example:

```
a=['a','b','c','d']
b=['a','b','c','d']
if(a is b):print('a and b have same identity')
else:print('a and b do not have same identity')
```

<u>The 'is not' Operator</u>

The 'is not' operator returns True, if the identity numbers of two objects being compared are not same. If they are same, then it will return False.

Example:

```
a=16
b=16
if(a is not b):print('a and b do not have same identity')
else:print('a and b have same identity')
```

Example:

```
a=['a','b','c','d']
b=['a','b','c','d']
if(a is not b):print('a and b do not have same identity')
else:print('a and b have same identity')
```

Input and Output Statements

The data given to the computer is called input. The results returned by the computer are called output. So, we can say that a computer takes input, processes that input and produces the output.

Output statements:

The output in Python is displayed using the print() function. The print() can be used in different formats.

The print() statement:

When the print function is called simply, it will throw the cursor to the next line. It means that a blank line will be displayed.

The print("String") statement

A string represents a group of characters. When a string is passed to the print() function, the string is displayed as it is. Example: print("Atmiya") #Atmiya

Note that single quotes or double quotes have the same meaning.

We can use escape sequence characters inside the print() function.

```
Print("This is the \n new line")
Print("The tab is \t used here")
```

The '+' operator when used with numbers will add both numbers. But when used with strings it will work as a concatenation operator. It will join two strings.

```
Print("The city is " + "Rajkot")
```

The print(variables list) statement

We can also display the values of variables using the print() function.

```
a,b=7,16
Print(a,b)
```

We can separate the string using 'sep' attribute.

```
Print(a,b,sep=",")
```

Each print() function throws the cursor to the next line after displaying the ouput.

```
Print('Atmiya')
Print('University')
Print('Rajkot')
```

If we want the output printed on the same line we can use 'end' attribute.

```
Print('Atmiya', end=' ')
Print('University')
Print('Rajkot')
```

We can use \t along with end parameter.

```
Print('Atmiya', end='\t ')
Print('University')
Print('Rajkot')
```

The print(object) statement

a=[7,'atmiya','rajkot']

Print(a)

<u>The print("string", variables list) statement</u>

The most common use of the print() function is to use strings along with variables inside the print().

```
a=7
print(a, "The value is printed here")
print("User has entered", a ,"as an input")
```

<u>The print(formatted string) statement</u>

We can format the output using the print() function. For that purpose the special operator '%' (percent) is used. It joins a string with a variable or value in the following format:

```
Print("Formatted string" % (variables list))
```

The formatted string could be %i or %d to represent decimal integer numbers. We can use %f to represent float values. We can use %s to represent strings.

To display single integer value:

```
a=7,
print('value=%i' %a)
```

To display more than one integer value:

```
a,b=10,16
print('a=%ib=%i' %(a,b))
```

We can use positive or negative values with %s to format the string.

```
university='Atmiya'
print('Hi %s'% university)

print('Hi %20s'% university)
print('Hi (%-20s)'% university)
```

We can use %c to display a single character

```
university='Atmiya'
print('The first character is %c, and the second character is %c'%(university[0],university[1]))
```

We can use %f to display the floating point values.

```
num=123.456789
print('The number is %f'%num)
print('The number is %8.2f'%num)
```

<u>Inside the formatted string</u>

```
no1,no2,no3=1,2,3
print('the number is {0}, the number is {1}, the number is {2}'.format(no1,no2,no3))

name,salary='ABC',16000
print('Hello {0}, your salary is {1}'.format(name,salary))
```

Input Statements

To accept input from the keyboard, Python provides the input() function. This function takes a value from the keyboard and returns it as a string.

```
a=input()
print(a)
#With message
name=input('Enter your name:')
print(name)
#Restricted to enter only integer
num=int(input('Enter the number'))
print(num)
#Restricted to enter only float
num=float(input('Enter the number'))
print(num)
#Only the first character will be stored in the variable and printed
char=input("Enter a character:")[0]
print(char)
#Only the third character will be stored in the variable and printed
char=input("Enter a character:")[2]
print(char)
#Finding sum and product of two integer numbers.
a=int(input("Enter the value"))
b=int(input("Enter the value"))
print('The sum of {0} and {1} is {2}'.format(a,b,a+b))
print('The sum of {0} and {1} is {2} and The product of {0} and {1} is {3}'.format(a,b,a+b,a*b))
```

<u>Use of split() method</u>

When the user enters three values, they are accepted as strings. These strings are divided wherever a space is found by split() method. These strings are read by for loop and converted into integers by the int() function. These integers are stored into no1, no2 and no3.

```
#Make use of split(), accept 3 values from user and make the sum.
no1,no2,no3=[int(no) for no in input("Enter three numbers by giving space in between: ").split()]
print("The sum of three values:", no1+no2+no3)
```

[or]

```
no1,no2,no3=[int(no) for no in input("Enter three numbers by giving space in between: ").split(,)]
print("The sum of three values:", no1+no2+no3)

#Accepting group of strings
a=[a for a in input('Enter strings:').split(',')]
print(a)
```

<u>Use of eval() function</u>

The eval() function takes a string and evaluates the result of the string by taking it as a Python Expression.

```
a,b=7,16
ans=eval("a+b-3")
print(ans)
```

We can use the eval() function along with input() function.

```
ans=eval(input("Enter an expression:"))
print('The result of your expression is: %d' %ans)
```

We can use combination of eval() and input() functions to accept objects like lists or tuples.

```
a=eval(input('Enter the values in list:'))
print("Values in the list are:",a)
```

Command Line Arguments

We can write a program which can accept the values while we are running our program in the command prompt. Command line arguments are the arguments which are passed to the program from outside the program. Arguments must be entered by keeping white space between them. The arguments passed at the command line are by default stored in the form of list with the name 'argv' which is the part of 'sys module'.

```
import sys
print(sys.argv) #run the code in command prompt, python file name. py 'a' 'b' 1 2 (7,16,25)
```

We can see that all the arguments are stored in the form of strings.

#Write a program to print the name of the python file you have executed.

```
import sys
a=sys.argv
print('The file name is:', a[0])
```

#Write a program to accept two values from the users at command line and add them.

```
import sys
a=int(sys.argv[1])
b=int(sys.argv[2])
sum=a+b
print('The sum of a and b is:', sum)
```

<u>If Statement</u>

The if statement is used to execute one or more statement depending on whether a condition is True or not.

Syntax:

```
if condition:
    statements
```

First, the condition is tested. If the condition is True, then the statements given after colon (:) are executed. We can write one or more statements after colon (:). If the condition is False, then the statements mentioned after colon are not executed.

Example:

```
a=1
if a==1:
    print('The value of a is 1')
```

Example:

```
a=int(input('Enter the value of a: '))
if a>7:
    print('The value of a is greater than 7')
```

Example:

```
a=int(input('Enter the value of a: '))
if a>0:
    print('The value of a is positive')
```

Example:

```
a='Atmiya'
if a=='Atmiya':
    print('The value of a is Atmiya')
    print('The value got matched')
    print('Thank you')
```

Example:

```
a=int(input('Enter the value of a: '))
b=int(input('Enter the value of a: '))
if a>b:
    print('The value of a is greater than value of b')
```

Example:

```
a=int(input('Enter the value of a: '))
b=int(input('Enter the value of b: '))
if a==1:
    print('The value of a is 1')
    if b==2:
        print('The value of b is 2')
print('End')
```

Example:

```
quantity = int(input('Enter the quantity: '))
price= 5
if price*quantity < 10000:
    print("The bill amount is less than Rs. 10000, we cannot dispatch your order")
    print("Please increase your quantity")
```

<u>The if...else statement</u>

The if...else statement executes a group of statements when a condition is True; otherwise, it will execute another group of statements.

Syntax:

```
if condition:
    statements1
else:
    statements2
```

Example:

```
a=int(input('Enter the value of a: '))
if a%2==0:
    print('The entered number is an Even number')
else:
    print('The entered number is an Odd number')
```

Example:

```
a=int(input('Enter the value of a: '))
if a==1:
    print('The value of a is 1')
else:
    print('The value of a is other than 1')
```

Example:

```
a=input('Enter the value of a: ')
if a=='Atmiya':
    print('The value of a is Atmiya')
    print('The value got matched')
else:
    print('The value of a is', a)
    print('The value does not matched')
```

Example:

```
a=int(input('Enter the value of a: '))
if a>7:
    print('The value of a is greater than 7')
else:
    print('The value of a is smaller than 7')
```

Example:

```
a=int(input('Enter the value of a: '))
b=int(input('Enter the value of a: '))
if a>b:
    print('The value of a is greater than value of b')
else:
    print('The value of a is smaller than value of b')
```

<u>The if...elif...else statement</u>

When there is a need of testing multiple conditions and execute statements depending on those conditions, you can write several elif statements between if...else, you can use if...elif...else is useful.

Syntax:

```
if condition1:
```

```
        statements1
elif condition2:
        statements2
elif condition3
        statements3
else:
        statements4
```

When condition1 is True, the statements1 will be executed. If condition1 is False, then condition2 is evaluated. When condition2 is True, the statements2 will be executed. When condition2 is False, the condition3 is tested. If condition3 is True, then statements3 will be executed. When condition3 is False, the statements4 will be executed. It means statements4 will be executed only if none of the conditions are True.

Example: Program to know if given number is positive, negative or zero

```
a=int(input('Enter the number: '))
if a>0:
        print(a, 'is a positive number')
elif a==0:
        print('It is a zero')
else:
        print(a, 'is a negative number')
```

Example: Writing else is not compulsory

Display entered numeric digit in words

```
a=int(input('Enter the number: '))
if a==0:
        print('Zero')
elif a==1:
        print('One')
elif a==2:
        print('Two')
elif a==3:
        print('Three')
elif a==4:
        print('Four')
elif a==5:
        print('Five')
```

If we try entering number other than these, it will not print anything. For displaying the message else must be used.

Example: Order>5000 than 10%, 8000 than 12%, 10000 than 15% 20000 than 25%

```
amount=int(input('Enter the total bill amount: '))
if amount<=0:
        print('Bill amount must be more than 0')
elif amount<=5000:
        print('Purchase little more to avail discount')
elif amount>=5000 and amount<=8000:
        print('You got 10 percent discount')
elif amount>=8000 and amount<=10000:
        print('You got 12 percent discount')
elif amount>=10000 and amount<=20000:
```

```
        print('You got 15 percent discount')
    elif amount>=20000:
        print('Hurray! You got highest discount of 25 percent')
```

Example:

```
    marks=int(input('Enter your marks: '))
    if marks<50:
        print('You need to re-appear the exam')
    elif marks>=50 and marks<60:
        print('You have passed the exam')
    elif marks>=60 and marks<70:
        print('You have passed the exam with First Class')
    elif marks>=70 and marks<80:
        print('You have passed the exam with Distinction')
    elif marks>=80 and marks<=100:
        print('You have passed the exam with Honors')
    else:
        print('Please enter valid marks')
```

Example: Extended example of order

```
    if amount<=0:
    print('Bill amount must be more than 0')
    elif amount<=5000:
        print('Purchase little more to avail discount')
    elif amount>=5000 and amount<=8000:
        print('You got 10 percent discount')
        discount=(amount*10)/100
        print('You saved', discount)
        pay=amount-discount
        print('Please pay only',pay)
        print('Thankyou for Shopping')
        print('Visit again')
```

<u>if vs. elif</u>

Example: (Using if) Accept marks of 4 students; display the student name that got highest marks

```
    std1=int(input('Please enter marks of Student 1: '))
    std2=int(input('Please enter marks of Student 2: '))
    std3=int(input('Please enter marks of Student 3: '))
    std4=int(input('Please enter marks of Student 4: '))
    if std1>std2 and std1>std3 and std1>std4:
        print('Student 1 got highest marks')
    if std2>std1 and std2>std3 and std2>std4:
        print('Student 2 got highest marks')
    if std3>std1 and std3>std2 and std3>std4:
        print('Student 3 got highest marks')
    if std4>std1 and std4>std2 and std4>std2:
        print('Student 4 got highest marks')
    else:
        print('Enter valid marks')
```

Example: (Using if…elif.else) Accept marks of 4 students; display the student name that got highest marks

```
std1=int(input('Please enter marks of Student 1: '))
std2=int(input('Please enter marks of Student 2: '))
std3=int(input('Please enter marks of Student 3: '))
std4=int(input('Please enter marks of Student 4: '))
if std1>std2 and std1>std3 and std1>std4:
        print('Student 1 got highest marks')
elif std2>std1 and std2>std3 and std2>std4:
        print('Student 2 got highest marks')
elif std3>std1 and std3>std2 and std3>std4:
        print('Student 3 got highest marks')
elif std4>std1 and std4>std2 and std4>std2:
        print('Student 4 got highest marks')
else:
   print ('The age must be a positive integer!')
```

Note: Multiple if's means your code would go and check all the if conditions, where as in case of elif, if one if condition satisfies it would not check other conditions.

Nested if:

```
a = 80
b = 40
c = 90
d=100
if(a>b):
   if(a>c):
        if(a>d):
                print("a is biggest")
if(b>a):
   if(b>c):
        if(b>d):
                print("b is biggest")
if(c>a):
   if(c>b):
        if(c>d):
                print("c is biggest")
if(d>a):
        if(d>b):
                if(d>c):
                        print("d is biggest")
```

The while loop

A normal statement is executed top to bottom only once. But we can use while loop to execute certain block of statements number of times depending on whether condition becomes True or False.

Syntax:

```
While condition:
        Statements
```

Here, statements represent one statement or block of statement. An interpreter first checks the condition. If the condition is True, then it will execute the statements written after the colon (:). After executing statements, it will go again to check condition. If the condition is found True, it will execute the statements. This process will be repeated as many times the condition becomes True. As soon as the condition becomes False, the interpreter will come out of the while loop.

Example: Program to display numbers from 1 to 10

```
a=1
while a<=10:
        print(a)
        #a=a+1
        a+=1
print('End of While Loop')
```

Example: Allow user to enter numbers, as soon as he press zero, program will give sum of all the entered numbers.

```
total = 0
number = int(input('Enter a number: '))
# add numbers until number is zero
while number != 0:
   total += number    # total = total + number
   # take integer input again
   number = int(input('Enter a number: '))
print('total =', total)
```

Example: Table of 1

```
num = 1
counter = 1
# we will use a while loop for iterating 10 times for the multiplication table
print("The Table of: ", num)
while counter <= 10: # specifying the condition
   ans = num * counter
   print (num, 'x', counter, '=', ans)
   counter += 1 # expression to increment the counter
```

Example: Program to display even numbers between 1 and 10

```
a=1
```

```
        while a>=1 and a<=10:
                print(a)
                a=a+2
```

Example: Even or odd based on user choice (1 for odd, 2 for even)

```
num=int(input('Enter 1 or 2 to get odd numbers or even numbers between 1-10: '))
if num==1:
        print('Program will print all odd numbers between 1-10')
        while num>=1 and num<=10:
                print(num)
                num=num+2
elif num==2:
        print('Program will print all odd numbers between 1-10')
        while num>=1 and num<=10:
                print(num)
                num=num+2
else:
        print('Enter either 1 or 2')
```

```
#Program to display number and their square
        #tonumber=2
        #fromnumber=20
        number=1
        while(number<=10):
                print(number,"\t", number**2)
                number=number+1
```

```
#Program to find and display only even numbers from the given set of numbers
        a=[2,4,5,8,3,7,8,4,2,34,15]
        b=0
        while(b<len(a)):
                if a[b]%2==0:
                        print(a[b])
                b=b+1
```

```
# a="Good Morning"
# len(a)
#Accept string from the user and print each character one by one
a='Atmiya'
b=0
while(b<len(a)):
        print(a[b])
        b=b+1
```

The For loop:

It can be used to execute a group of statements repeatedly depending upon the number of elements in the sequence. The for loop can work with sequence like string, list, tuple, range etc.

Syntax:

```
For var in sequence:
        Statements
```

The first element of the sequence is assigned to the variable written after 'for' and then the statements are executed. Next, the second element of the sequence is assigned to the variable and the statements are executed second time. In this way, for each element of the sequence, the statements are executed once. So, the for loop is executed as many times as there are number of elements in the sequence.

Example: Printing elements of the string

```
a='Atmiya'
for ch in a:
        print('\t',ch)
```

Example: Display elements of the string using index

```
a='Atmiya'
n=len(a)
for i in range(n):
        print(a[i])
```

The range() function in Python is useful to provide a sequence of numbers. It gives number from 0 to n-1.

Example:Display 0-9

```
for i in range(10):
print (i)
```

Example: Print odd numbers between 1-10

```
for i in range(1,11,2):
print (i)
```

Example: Print even numbers between 1-10

```
for i in range(2,11,2):
print (i)
```

Example: Print 10-1

```
for i in range(10,0,-1):
print (i)
```

Example: Printing each elements from the list one by one

```
lst=['a',7,16,25,'abc',13.3,'Atmiya','India']
for i in lst:
        print(i)
```

Example: Add all the elements of the list

```
lst=[2,5,6,7,59,5]
sum=0
for i in lst:
```

```
        print(i)
        sum=sum+i
print('The sum is: ', sum)
```

Example: Using else with for

```
lst=['a',7,16,25,'abc',13.3,'Atmiya','India']
for i in lst:
        print(i)
else:
        print('All the elements are printed')
```

#Finding square of each element of the list

```
numbers = [3, 5, 23, 6, 5, 1, 2, 9, 8]
sqr=0
n=len(numbers)
for i in range(n):
        #print(numbers[i])
        sqr=numbers[i]**2
        print(sqr)
```

#Finding sum of square of each element of the list

```
numbers = [3, 5, 23, 6, 5, 1, 2, 9, 8]
sum=0
n=len(numbers)
for i in range(n):
        #print(numbers[i])
        sum=sum+numbers[i]**2
print(sum)
```

Nested Loops

Nested loops mean loops inside a loop. In Python programming language there are two types of loops which are for loop and while loop. Using these loops we can create nested loops. For example, while loop inside the for loop, for loop inside the for loop, etc. The "inner loop" will be executed one time for each iteration of the "outer loop".

Structure:

```
Outer loop:
        Inner loop:
```

Example:

```
for i in range(2):
        for j in range(3):
                print('i=',i, '\t','j=',j)
```

Example:

```
for i in range(4):
        print(i,end="")
```

Example:

```
for i in range(3):
        for j in range(4):
```

```
            print(j,end="")
        print()
```

Note: Outer loop is in-charge of rows; inner loop is in-charge of columns.

The break statement

The break statement in Python works as a loop control statement. It can be used inside a for loop, while loop or nested loop to come out of the loop. When break statement is executed, the Python interpreter jumps out of the loop to process the next statement in the program. Break statement is used to bring the control out of the loop when some external condition is triggered. It terminates the current loop, i.e., the loop in which it appears, and resumes execution at the next statement immediately after the end of that loop. If the break statement is inside a nested loop, the break will terminate the innermost loop.

Example: Take input from the user, check whether the inputed number is present in the list or not.

```
a=[1,2,3,4,5]
search=int(input('Enter an element to search: '))
for ele in a:
    if search==ele:
        print('Element found in the list')
        break
else:
    print('Element is not available in the list')
```

Example: Print 10 to 5

```
a=10
while a>=1:
    print (a)
    a=a-1
    if a==4:
        break
print('Out of loop')
```

Example: Using break with string

```
a='Atmiya'
for b in a:
    print(b)
    if b=='y':
        break
```

Example: Same program as above using while

```
a='Atmiya'
i=0
while True:
    print(a[i])
    if a[i]=='y':
        break
    i=i+1
```

```
#Using break in nested loop
l1=[2,3,4,5]
l2=[7,8,9,10]
for i in l1:
    for j in l2:
        print(i,j)
        if i==4 and j==9:
            print('break')
            break
```

The continue statement

Example:

```
a='AtmiyaUniversityRajkot'
for i in a:
    if i=='a':
        continue
print(i,end='')
```

Example: Print 1-10, except 5 and 6

```
for i in range(1,11):
    if i==5 or i==6:
        continue
else:
    print(i,end='')
```

The Pass statement

The pass statement is used as a placeholder for future code. When the pass statement is executed, nothing happens, but you avoid getting an error when empty code is not allowed. Empty code is not allowed in loops, function definitions, class definitions, or in if statements. Using pass inside a loop, function or a class gives you a facility to keep it empty. Pass statement is used when we need a statement syntactically but we do not want to do any operation. The difference between pass and comment is that comment is ignored by the interpreter whereas pass is not ignored.

Example: Pass does nothing

```
a=0
while a<10:
    a=a+1
    if a>5:
        pass
    print (a)
```

Example: Program to display only negative numbers from the list

```
a=[1,2,4,-4,3,-1,5,-10]
for i in a:
    if (i>0):
        pass
    else:
        print(i)
```